Transformer-Based Abstractive Text Summarizer

Complete Project Study Guide & Interview Preparation

Architecture • Training • Classical NLP • Baselines • Evaluation • ML Suite • Deployment • Interview Q&A

IIT Hyderabad | B.Tech Project

1. Project Overview

This project implements an abstractive text summarization system entirely from scratch in PyTorch — no HuggingFace, no pre-trained weights. Given a news article, the model generates a short novel summary (new words, not copied sentences). The full pipeline covers data preprocessing, a Transformer encoder-decoder, classical NLP baselines, ML-based evaluation, and a 10-page Streamlit dashboard deployed on Streamlit Cloud.

Abstractive vs Extractive Summarization

Extractive: Picks and copies the most important existing sentences. No new words. Example: TF-IDF baseline in this project.

Abstractive: Generates new sentences that paraphrase the source — like a human writer. Words can appear in the summary that never appeared in the article. Example: our Transformer model.

Dataset: CNN/DailyMail

- 287,000 article-summary pairs from CNN and DailyMail news websites.
- Articles: ~800 words average. Summaries: ~50 words average.
- Vocabulary: top 8,000 most frequent tokens. Special tokens: PAD=0, UNK=1, SOS=2, EOS=3.
- Any token not in the vocabulary is replaced with UNK at inference time.

High-Level Pipeline

- Step 1: Raw text -> tokenize -> vocabulary -> integer sequences.
- Step 2: Batch + pad sequences -> Transformer encoder.
- Step 3: Encoder produces contextual representation of every source token.
- Step 4: Decoder auto-regressively generates summary tokens using cross-attention over encoder output.
- Step 5: Train with teacher forcing. Inference with greedy decoding.
- Step 6: Evaluate with n-gram overlap, sentiment, NER, topic classification.

2. Classical NLP Preprocessing Pipeline

2.1 Tokenization

Splits raw text into a list of tokens (words). Our tokenizer splits on whitespace and punctuation, lowercases all text. Each unique token gets an integer ID from the vocabulary.

In this project: *Custom word tokenizer — no NLTK or spaCy for tokenization.*

2.2 Vocabulary Building

Count frequency of every token in training corpus. Sort by frequency. Keep top 8,000. Add special tokens. Any token outside the vocabulary maps to UNK at runtime.

2.3 Stopword Removal

Stopwords are very common words (the, a, is, on...) carrying little meaning for scoring or classification. We maintain a hardcoded list of ~100 English stopwords.

In this project: *Used in TF-IDF baseline scoring and as preprocessing for ML classifiers.*

2.4 Stemming — Porter Stemmer

Reduces a word to its root by stripping suffixes using rule-based phases: running->run, happiness->happi. The Porter Stemmer has 5 phases of suffix-stripping rules. Fast but can produce non-words. Implemented entirely from scratch.

2.5 Lemmatization

Reduces a word to its dictionary base form: running->run, better->good. Always produces a real word. Our rule-based lemmatizer uses suffix replacement rules and an exceptions dictionary without requiring a POS tagger.

In this project: *Both stemmer and lemmatizer demonstrated in the Streamlit NLP Preprocessing Explorer page.*

2.6 N-grams

A contiguous sequence of n tokens. Unigram=1 word, Bigram=2 adjacent words, Trigram=3 consecutive words. Example: 'The cat sat' -> bigrams: {The cat, cat sat}. Used in evaluation to measure overlap between generated and reference summaries.

2.7 Bag of Words (BoW)

Represents a document as a vector of word counts, ignoring word order. Vocabulary size = vector dimension. Simple but loses all sequential information. Used as features for ML classifiers alongside TF-IDF.

2.8 TF-IDF

Weights a word by how often it appears in a document (TF) discounted by how common it is across all documents (IDF):

- TF(word, doc) = count(word in doc) / total words in doc
- IDF(word) = log((1+N) / (1+df(word))) + 1 [smoothed]
- TF-IDF = TF x IDF. Common words like 'the' get low IDF; distinctive words get high TF-IDF.

In this project: *Built from scratch. Corpus IDF computed over training articles. Used for: (1) TF-IDF extractive baseline sentence scoring. (2) Feature vectors for Logistic Regression and Random Forest.*

3. Transformer Architecture — Deep Dive

The Transformer ('Attention Is All You Need', Vaswani et al. 2017) replaces recurrence entirely with attention mechanisms. This project implements every component from scratch: 11.68M parameters, d_model=256, 4 heads, d_ff=1024, 3 encoder + 3 decoder layers.

3.1 Token Embeddings

Each integer token ID maps to a dense learnable vector of size d_model=256 via an embedding matrix (vocab_size x d_model). The embedding is scaled by sqrt(d_model) to balance its magnitude against the positional encoding. This scaling stabilizes training.

3.2 Sinusoidal Positional Encoding

Transformers have no built-in sense of word order — all tokens are processed in parallel. Positional encoding injects position information by adding a fixed vector to each token embedding. Formulas:

- PE(pos, 2i) = sin(pos / 10000^(2i/d_model))
- PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
- Fixed (no parameters to learn). Generalizes to unseen sequence lengths. Relative positions can be expressed as linear combinations.

In this project: *Pre-computed PE matrix added to embeddings at start of encoder and decoder.*

3.3 Scaled Dot-Product Attention

Core attention: Attention(Q,K,V) = softmax(Q x K^T / sqrt(d_k)) x V

- Q x K^T computes similarity score between every query and every key (all pairs).
- Divide by sqrt(d_k) prevents dot products from growing large -> prevents flat softmax -> healthy gradients.
- softmax converts scores to probabilities (attention weights summing to 1).
- Multiply by V: output is weighted sum of values — positions with high attention weight contribute more.

3.4 Multi-Head Attention

Run h=4 parallel attention heads, each with its own linear projections of Q, K, V. Each head attends to different relationship types (syntax, semantics, co-reference). Outputs concatenated and projected back to d_model.

- d_k = d_model / h = 256/4 = 64 per head.
- W_Q, W_K, W_V: each d_model x d_k. W_O: (h*d_v) x d_model.
- MultiHead(Q,K,V) = Concat(head_1,...,head_h) x W_O

3.5 Masked Self-Attention (Decoder)

In the decoder, each position can only attend to earlier positions (causal masking) — prevents cheating by looking at future summary tokens during training. The mask is an upper-triangular matrix of -infinity values added before softmax, making future positions have zero probability after softmax.

3.6 Cross-Attention (Encoder-Decoder Attention)

The decoder's second sub-layer attends over the encoder's output. Q comes from the decoder; K and V come from the encoder. This is how the decoder reads the source article while generating the summary. Without cross-attention,

the decoder has no access to the source.

3.7 Position-wise Feed-Forward Network (FFN)

After attention, each position passes through: FFN(x) = max(0, x*W1 + b1)*W2 + b2. Inner dim d_ff=1024 (4x d_model). ReLU activation. Adds non-linearity and per-position transformation capacity. Same weights shared across positions, different across layers.

3.8 Residual Connections + Layer Normalization

Residual: output = LayerNorm(x + SubLayer(x)). Gradient highway — gradients flow directly backward without passing through SubLayer. Solves vanishing gradient. Makes learning identity easier (learn the delta, not the full function).

LayerNorm: Normalizes across the feature dimension (d_model) for each position independently. LN(x) = gamma * (x - mean) / sqrt(var + eps) + beta. Unlike BatchNorm (across batch), LayerNorm works on a single example — better for variable-length sequences.

3.9 Encoder Block (x3)

Each encoder layer: (1) Multi-Head Self-Attention — every source token attends to all others. (2) FFN. Each wrapped with Residual + LayerNorm. Three stacked layers build progressively richer source representations.

3.10 Decoder Block (x3)

Each decoder layer: (1) Masked Self-Attention — attend to previously generated tokens only. (2) Cross-Attention — attend over encoder output to read source. (3) FFN. All wrapped with Residual + LayerNorm.

3.11 Generator (Output Layer)

Final linear projection: d_model -> vocab_size=8,000 logits. Softmax gives probability distribution over vocabulary. Training: cross-entropy loss against true next token. Inference: argmax (greedy decoding).

Parameter Count Breakdown

- Token embedding: 8000 x 256 = 2.05M
- 3 encoder layers (~790K each) = 2.37M
- 3 decoder layers (~1.05M each) = 3.15M
- Output projection: 256 x 8000 = 2.05M
- Layer norms, biases: ~1.06M
- Total: ~11.68M parameters

4. Training Pipeline

4.1 Loss: Cross-Entropy with Label Smoothing

At each decoder step, model predicts probability over 8,000 tokens. Loss = -log(P(true_token)). Label smoothing (eps=0.1) distributes 0.1 probability mass across all tokens instead of one-hot targets — prevents overconfidence, improves generalization.

4.2 Backpropagation

PyTorch autograd computes gradients of loss w.r.t. every parameter via chain rule. Gradients flow: generator -> decoder layers (attention + FFN) -> cross-attention -> encoder layers -> embeddings.

4.3 Adam Optimizer

Adaptive Moment Estimation. Tracks two moments per parameter:

- m = beta1*m + (1-beta1)*grad (first moment: gradient direction, beta1=0.9)
- v = beta2*v + (1-beta2)*grad^2 (second moment: gradient magnitude, beta2=0.98)
- Bias-corrected: m_hat = m/(1-beta1^t), v_hat = v/(1-beta2^t)
- Update: theta = theta - lr * m_hat / (sqrt(v_hat) + eps), eps=1e-9
- Adaptive: parameters with large past gradients get smaller effective LR. Ideal for sparse gradients in attention.

4.4 Learning Rate Scheduling

Two-phase schedule: Phase 1 (warmup_steps): LR increases linearly. Phase 2: LR decreases as inverse square root of step. Formula: LR = d_model^(-0.5) * min(step^(-0.5), step * warmup_steps^(-1.5)). Warmup prevents large destructive updates when gradients are noisy at the start.

4.5 Teacher Forcing

During training, feed the true ground-truth token as decoder input at each step (not model's own prediction). Speeds up convergence. Exposure bias: at inference the model sees its own predictions, not ground truth — can cause error accumulation.

4.6 Xavier Initialization

Weights sampled from Uniform(-limit, +limit) where limit = sqrt(6 / (fan_in + fan_out)). Keeps variance of activations and gradients roughly equal across layers. Prevents vanishing/exploding gradients at training start.

4.7 Dropout (p=0.1)

Randomly zeros activations during training with probability p. Forces redundant representations. Acts as regularization. Disabled at inference. Applied after attention and FFN sub-layers.

4.8 Gradient Clipping (max_norm=1.0)

If the L2 norm of all gradients exceeds 1.0, rescale them: g = g * (1.0 / ||g||). Prevents exploding gradients from destroying learned weights. Especially important for sequences where gradient products can grow exponentially.

4.9 Results

• Training loss: 5.76 -> 3.04 (47% reduction, 10 epochs).

• Validation loss: 4.77 -> 3.62. Gap = mild overfitting controlled by dropout.

• ~50 minutes total training time.

• Evaluated on 200 held-out test examples.

5. Baseline Models

5.1 TF-IDF Extractive Baseline

Classical extractive approach — selects, does not generate.

- Score each sentence: sum TF-IDF weights of its content words (after stopword removal).
- Select top-k sentences by score as the summary.
- Corpus IDF computed over all training articles for global word importance weighting.

In this project: *Built entirely from scratch. No sklearn. Serves as the extractive baseline.*

5.2 Bidirectional LSTM + Bahdanau Attention

Bidirectional LSTM Encoder: Two LSTMs run in parallel — one forward (left to right), one backward (right to left). Hidden states concatenated. Each position has context from both directions. LSTMs use 4 gates (input, forget, output, cell state) to control long-range information flow, partially solving vanishing gradients of vanilla RNNs.

Bahdanau (Additive) Attention: At each decoder step, compute alignment score between decoder state s and each encoder state h: score(s,h) = v^T * tanh(W_s*s + W_h*h). 'Additive' because it concatenates s and h before projecting (vs Transformer's multiplicative dot-product). First attention mechanism for NLP (Bahdanau 2014).

Comparison with Transformer

- Transformer unigram F1: 24.5% vs TF-IDF: 17.0% -> Transformer +44%.
- Transformer LCS F1: 19.9% vs TF-IDF: 9.2% -> Transformer +117%.
- TF-IDF: high precision (exact copies) but low recall (misses paraphrased content).
- Transformer: generates novel phrasing — more abstractive, lower n-gram overlap but more faithful semantically.
- BiLSTM+Bahdanau: middle ground — attentive generation but weaker than full Transformer.

6. Evaluation Suite

6.1 N-gram Overlap (from scratch)

- Precision = |overlap| / |generated n-grams|
- Recall = |overlap| / |reference n-grams|
- F1 = 2*P*R / (P+R)
- Unigram F1=24.5%, Bigram F1=3.7%, LCS F1=19.9% (evaluated on 200 test pairs).
- LCS (Longest Common Subsequence) does not require consecutive matches — more flexible than n-gram.

6.2 Intrinsic vs Extrinsic Evaluation

Intrinsic: Measures output quality directly against a reference (n-gram overlap, NER). **Extrinsic:** Measures impact on a downstream task (which model's summaries better preserve sentiment and topic). We use both.

6.3 Sentiment Consistency — Logistic Regression

Pseudo-labels from sentiment lexicon. TF-IDF feature vectors. LR trained with binary cross-entropy + L2 regularization. Predict sentiment of article and summary; consistency = fraction of matching pairs. Transformer: 80% vs TF-IDF: 60%.

LR Theory: P(y=1|x) = sigmoid(w^T x + b). sigmoid: f(z) = 1/(1+e^(-z)). Loss = -[y*log(p) + (1-y)*log(1-p)]. Gradient: dL/dw = (p-y)*x. L2 reg adds lambda*||w||^2 to loss.

6.4 Topic Classifier — Random Forest

Decision Tree: Splits data on the feature maximally reducing impurity. Entropy H(S) = -sum(p_i * log2(p_i)). Information Gain = H(parent) - weighted_avg(H(children)). Gini = 1 - sum(p_i^2). We use entropy splits.

Random Forest: 30 trees, each trained on a bootstrap sample (random sample with replacement). Each split considers random feature subset. Final prediction = majority vote. Bootstrapping + feature randomness ensure uncorrelated trees — averaging cancels errors.

5-fold CV: Split into 5 folds. Train on 4, test on 1. Rotate 5 times. Average metrics. More reliable than single train/test split.

6.5 NER Entity Preservation

spaCy identifies named entities (persons, orgs, locations). Rule-based fallback: capitalized multi-word noun phrases. Measures entity recall (how many source entities appear in summary). TF-IDF: recall=53.6%, precision=100%, F1=66.7%. Transformer: 0% (lowercase output prevents rule-based NER detection).

7. Streamlit Dashboard

10-page interactive web app on Streamlit Cloud. Streamlit renders Python as a web app. Plotly provides interactive charts.

- Home: Overview and navigation.
- Summarize: Live inference — enter article, get Transformer summary.
- Attention Visualization: Plotly heatmap — rows=summary tokens, cols=source tokens, color=attention weight.
- NLP Preprocessing: Interactive explorer for tokenization, stemming, lemmatization, TF-IDF, BoW.
- Results: N-gram metrics table, per-topic breakdown.
- About: Architecture + tech stack.
- Baseline Comparison: Side-by-side Transformer vs TF-IDF vs BiLSTM.
- Sentiment Analysis: Live classifier demo + corpus consistency scores.
- Topic Classifier: Per-topic performance.
- NER Analysis: Entity preservation metrics.

8. Interview Questions & Answers

Format: Q (interviewer question) -> Answer -> Follow-up counter questions + answers.

A. Project Walk-Through Questions

Q: Walk me through your project end to end.

I built an abstractive text summarization system entirely from scratch in PyTorch on the CNN/DailyMail dataset (287K pairs). I implemented a custom tokenizer and vocabulary (8K tokens), then built a Transformer encoder-decoder: encoder reads the article creating contextual representations; decoder generates summary word by word using cross-attention over encoder output. Trained 10 epochs with Adam + learning rate scheduling, achieving 47% training loss reduction (5.76->3.04). For evaluation I built n-gram overlap metrics from scratch (unigram F1=24.5%), a Logistic Regression sentiment consistency classifier (80% vs 60% TF-IDF), a Random Forest topic classifier (5-fold CV), and NER preservation analysis. Also implemented two baselines: TF-IDF extractive and BiLSTM+Bahdanau. All deployed on a 10-page Streamlit dashboard.

Follow-up: Why abstractive instead of extractive?

Abstractive is harder but more natural — a human summary paraphrases, it doesn't just copy sentences. The project goal was to understand the full Transformer pipeline, which is inherently generative.

Follow-up: What was the hardest part?

Debugging the decoder's causal mask. An off-by-one in the upper-triangular mask let the decoder peek at future tokens during training, causing repetitive looping outputs at inference. Fixing the mask shape resolved it immediately.

Q: What does 11.68M parameters mean?

Each learnable number in the model is a parameter. 11.68 million means there are 11.68M floating point values that are adjusted during training. The largest contributors: token embedding matrix (8000x256=2.05M), output projection (256x8000=2.05M), encoder attention + FFN layers (~2.37M), decoder layers (~3.15M). More parameters = more capacity to learn patterns, but also more data and compute needed.

Follow-up: Is 11.68M large or small?

Small by modern standards. GPT-3 has 175B parameters; BERT-base has 110M. But for a from-scratch project on a single GPU training on 287K pairs, 11.68M is well-sized — enough capacity to learn meaningful summarization patterns without requiring weeks of training.

Q: Why 8,000 vocabulary tokens?

A larger vocabulary covers more words (fewer UNK replacements) but increases embedding matrix size and output projection size quadratically in memory. 8K covers ~95% of token occurrences in CNN/DailyMail. The remaining 5% are rare proper nouns and domain-specific terms that map to UNK. 8K is a practical balance for from-scratch training on modest hardware.

Follow-up: What would BPE (Byte Pair Encoding) do better?

BPE tokenizes rare words into subword units rather than replacing with UNK. 'Hyderabad' -> 'Hyder' + 'abad'. This gives better coverage with the same vocabulary size and is why modern models (GPT, BERT) use BPE. It would fix the NER issue where proper nouns get UNKed.

B. Architecture Theory Questions

Q: What is self-attention? Why is it better than RNNs for NLP?

Self-attention lets every token attend to every other token in one operation. Each token creates a Query vector; all tokens create Key and Value vectors. Output for each token = weighted sum of all Values, weights from softmax(Q*K^T / sqrt(d_k)). Captures long-range dependencies in O(1) operations vs RNNs which propagate information step by step — making long-range learning hard due to vanishing gradients. Also fully parallelizable across the sequence, enabling much faster training on modern hardware.

Follow-up: Time complexity of self-attention?

O(n^2 * d) — quadratic in sequence length n. For n=800 tokens this is fine. For very long documents (books, legal contracts) the quadratic cost is prohibitive — motivating sparse attention variants like Longformer.

Follow-up: Does attention replace word order information?

No — attention is permutation-invariant by itself. Positional encoding is explicitly added to embeddings before attention to inject order. Without PE, the model would treat 'cat chases dog' and 'dog chases cat' identically.

Q: Why do we divide by sqrt(d_k) in attention?

The dot product Q*K^T is a sum of d_k multiplications. As d_k increases, the expected magnitude of the dot product grows as sqrt(d_k). Large dot products push softmax into near-zero gradient regions (output becomes nearly one-hot, small gradients everywhere else). Dividing by sqrt(d_k) normalizes the magnitude back to O(1) regardless of d_k, keeping gradients healthy throughout training.

Follow-up: What if we used a smaller temperature (divide by more)?

More smoothing — attention becomes more uniform across positions. Useful for exploration early in training. Too much smoothing and the model can't focus on relevant tokens. Temperature is a hyperparameter that can be tuned.

Q: What is the difference between self-attention and cross-attention?

In self-attention, Q, K, V all come from the same sequence (encoder reads itself; decoder reads its own history). In cross-attention, Q comes from the decoder but K and V come from the encoder output. The decoder queries the source article at every generation step, aligning each generated word with relevant source content. This is the information bridge between encoder and decoder.

Follow-up: Why does the decoder need both self-attention AND cross-attention?

Masked self-attention builds context from previously generated tokens (maintains coherent output so far). Cross-attention reads the source (knows what to say). Without self-attention: incoherent output. Without cross-attention: decoder ignores the source entirely.

Q: Explain residual connections. Why are they important?

output = LayerNorm(x + SubLayer(x)). The input x bypasses the sub-layer and is added directly to its output. During backpropagation, gradients can flow straight back through the addition (+) without passing through SubLayer — this is a gradient highway preventing vanishing gradients in deep networks. It also makes learning easier: the sub-layer only needs to learn the residual (change from input), not the full function from scratch.

Follow-up: What problem do residuals solve for Transformers specifically?

Transformers have 6-12+ layers. Without residuals, gradients passing through that many attention + FFN operations would vanish to near-zero by the first layer, making it untrainable. Residuals let even the earliest layers receive meaningful gradient signal.

C. Training Theory Questions

Q: Why Adam over SGD?

Adam maintains per-parameter adaptive learning rates via two moment estimates. This handles the highly variable gradient magnitudes in Transformers: embedding rows for rare words see infrequent but large gradients; attention weights see frequent but small gradients. Adam naturally adapts to each parameter's history. SGD applies the same LR to all parameters, requiring extensive tuning to work well on Transformers. In practice, Adam (or AdamW) is the standard for Transformer training.

Follow-up: What is AdamW?

AdamW decouples L2 weight decay from the gradient update — in vanilla Adam, weight decay interacts with the adaptive LR scaling in unintuitive ways. AdamW applies weight decay directly to the parameters, which is cleaner and often gives better generalization. Modern best practice is AdamW.

Q: What is teacher forcing and what is exposure bias?

Teacher forcing: during training, the decoder receives the true ground-truth token as input at each step (not its own prediction). This prevents early training errors from compounding and speeds convergence. Exposure bias: the resulting mismatch between training (always sees real tokens) and inference (sees its own predictions). If an early predicted token is wrong, all subsequent tokens are conditioned on wrong context — errors accumulate. Scheduled sampling (gradually replacing ground truth with model predictions during training) can help.

Follow-up: How does your model decode at inference?

Greedy decoding: take argmax of output probability distribution at each step. Simple and fast but not optimal — the globally best sequence might require a locally suboptimal token early on. Beam search (maintain top-k partial hypotheses) finds better sequences at the cost of k times the compute.

Q: Explain gradient clipping. Why is it specifically important for Transformers?

Gradient clipping: if ||gradients|| > max_norm=1.0, scale all gradients by max_norm/||gradients||. This prevents a single large gradient update from overwriting learned weights. Transformers are susceptible to exploding gradients because attention weights create multiplicative paths through many layers — a large gradient can grow exponentially through those multiplications. Clipping bounds the update magnitude without stopping learning.

Follow-up: How is clipping different from a small fixed LR?

A small LR slows down ALL updates, even small beneficial ones. Clipping only intervenes when gradients are unusually large — normal updates are unaffected. Clipping + LR scheduling gives the benefits of both: fast normal updates, bounded catastrophic updates.

Q: Why did training loss stop at 3.04?

Cross-entropy loss of 3.04 means the model assigns average probability e^(-3.04) ~ 4.8% to the correct next token over an 8,000-word vocabulary (random would be 0.0125%). This is reasonable for abstractive summarization — many valid next tokens exist at each step, so perfect prediction is impossible. Reasons for plateau: (1) Model capacity limit at 11.68M parameters. (2) CNN/DailyMail has highly variable reference summaries (many valid summaries per article). (3) Only 10 training epochs. (4) 8K vocabulary with UNK replacing rare words adds noise.

Follow-up: How would you push loss lower?

Larger model (6 layers, d_model=512). More training epochs. BPE subword tokenization. Pre-trained embeddings. Learning rate tuning. Data augmentation. Beam search at inference (doesn't affect training loss but improves output quality).

D. Classical NLP Questions

Q: What is TF-IDF and how did you use it in the evaluation suite?

TF-IDF scores word importance: TF(w,d) = count(w in d)/doc_length. IDF(w) = log((1+N)/(1+df(w)))+1 where N=corpus size, df=documents containing w. Common words have high TF everywhere but very low IDF (so low TF-IDF). Distinctive words have high TF in few documents and high IDF (so high TF-IDF). Uses in this project: (1) Extractive baseline: score sentences by TF-IDF word sum, pick top-k. (2) Feature engineering: TF-IDF vectors fed to Logistic Regression (sentiment) and Random Forest (topic classifier).

Follow-up: Why not use word embeddings as features for LR/RF?

TF-IDF is interpretable — each dimension corresponds to a known word. For small datasets, TF-IDF+LR/RF often outperforms embeddings. Embeddings require sufficient training data to learn meaningful representations; TF-IDF is completely deterministic and effective for topic/sentiment classification on news text.

Q: Explain the difference between stemming and lemmatization.

Stemming: rule-based suffix removal producing a word root — fast but may produce non-words ('happiness'->'happi'). Porter Stemmer uses 5 phases of suffix rules, e.g., 'sses' -> 'ss', 'ational' -> 'ate'. Lemmatization: looks up the canonical base form (lemma) — always a real word ('better'->'good', 'running'->'run') but slower and may need POS context. Both reduce vocabulary size and group morphological variants for better feature matching.

Follow-up: When would you prefer stemming over lemmatization?

When speed is critical and downstream task tolerates approximate matching (e.g., search indexing). Lemmatization preferred when linguistic accuracy matters (NLP analysis, careful NER, grammar checking). For TF-IDF feature engineering in classifiers, both work similarly well.

Q: What is NER and why did the Transformer get 0% entity recall?

NER identifies real-world entities: persons, organizations, locations, dates, etc. spaCy's statistical model identifies entity spans; our rule-based fallback finds capitalized multi-word noun phrases. The Transformer outputs all-lowercase text because our vocabulary was built from lowercased tokens — during inference, all output tokens are lowercase. The rule-based NER requires capitalization to identify proper nouns, so it finds nothing in lowercase output. This is a preprocessing artifact, not a model quality failure. Using spaCy (which handles lowercase via learned context) would give real entity recall scores.

Follow-up: How would you fix this?

Two options: (1) Build the vocabulary with case-preserved tokens — more expensive but preserves entity names. (2) Use spaCy for NER evaluation which does not require capitalization — it uses learned statistical patterns that work on lowercase text.

E. Machine Learning Questions

Q: How does Logistic Regression work as a sentiment classifier?

LR learns a linear decision boundary. Each TF-IDF feature (word) gets a weight w_i. Prediction: P(positive|x) = sigmoid(sum(w_i * x_i) + b). Since CNN/DailyMail has no sentiment labels, we pseudo-label: count positive-sentiment words (success, win, praised) vs negative (killed, disaster, crime); assign majority label. LR minimizes binary cross-entropy loss: L = -[y*log(p) + (1-y)*log(1-p)]. Gradient update: w -= lr * (p-y) * x. L2 regularization: w -= lr * lambda * w (keeps weights small).

Follow-up: Why L2 regularization?

Without regularization, LR can assign extreme weights to discriminative words that appear in training but not test (overfitting). L2 penalizes large weights, distributing importance across many moderate-weight features — generalizes better to unseen text.

Follow-up: Why not use a neural network for sentiment?

For small pseudo-labeled datasets, LR is more stable than neural networks and gives interpretable weights (high w_i for 'success' confirms positive association). Neural networks would need more labeled data to outperform LR.

Q: Explain Random Forest. How does bagging reduce overfitting?

A decision tree can overfit by growing deep and memorizing training noise. Random Forest builds 30 trees, each on a different bootstrap sample (sample n examples with replacement from n training examples — about 63.2% unique). Each split also considers a random feature subset. Predictions are majority-voted. Bagging works because: (1) Each tree sees different data -> different errors. (2) The errors are approximately uncorrelated. (3) Averaging uncorrelated errors reduces variance without increasing bias. Net effect: individual trees have high variance, ensemble has low variance.

Follow-up: What is the difference between bagging and boosting?

Bagging: trees trained independently in parallel on random subsets; average reduces variance. Boosting (AdaBoost, XGBoost): trees trained sequentially, each focusing on examples previous trees got wrong; reduces both bias and variance but can overfit and is slower to train.

Follow-up: Why 30 trees specifically?

30 is a reasonable default providing stable predictions without excessive compute. Performance generally improves up to ~100-200 trees then plateaus. We chose 30 to keep evaluation fast; in production you'd tune this via cross-validation.

Q: What is 5-fold cross-validation and when would you NOT use it?

Split data into 5 equal folds. In 5 rounds, train on 4 folds, validate on 1 (rotating which fold is held out). Average metrics across rounds. Benefits: (1) Every example used for both training and validation. (2) More reliable estimate than single split. (3) Provides variance estimate across folds. DO NOT use for: (1) Time-series data — future information would leak into past training folds (use forward-chaining instead). (2) Very large datasets — single split is reliable enough. (3) When data has inherent structure (grouped by patient/user) — stratify folds by group.

Follow-up: What is stratified k-fold?

Ensures each fold has the same class distribution as the full dataset. Important for imbalanced classes — without stratification, some folds might have very few minority-class examples, making evaluation noisy.

F. Evaluation Questions

Q: Why did you implement ROUGE from scratch instead of using a library?

Two reasons: (1) Consistency: the standard ROUGE library uses its own tokenization and normalization that differs from our training vocabulary. Using the library would measure a different token distribution than we trained on. Building from scratch uses the same tokenizer as training. (2) Understanding: implementing n-gram F1 from scratch forces deep understanding of what ROUGE actually measures — not a black box. The math is: count overlapping n-grams, compute precision/recall/F1 — exactly what we implemented.

Follow-up: What are ROUGE's limitations for abstractive summarization?

ROUGE rewards exact word matches. An abstractive model that correctly paraphrases using different words scores low. A model that copies verbatim scores high even if it picks the wrong sentences. For abstractive evaluation, BERTScore (semantic similarity via contextual embeddings) or human evaluation are more appropriate.

Q: Your bigram F1 is only 3.7% while unigram is 24.5%. What does this tell you?

The model uses the right vocabulary (24.5% of individual words overlap with reference) but generates them in different syntactic combinations (only 3.7% of consecutive word pairs match). This is the hallmark of abstractive generation — the model paraphrases rather than copying. A purely extractive system would have much higher bigram F1 because it copies exact sentences. The gap confirms the model is truly generating novel phrasing rather than memorizing and copying training examples.

Follow-up: Is low bigram F1 always bad?

Not necessarily for abstractive summarization. It means the model generates novel text, which is the goal. The relevant question is whether the generated text is semantically correct (covered by sentiment consistency and NER preservation metrics) not just lexically similar.

G. Design Decisions

Q: Why build everything from scratch instead of using HuggingFace?

HuggingFace gives you a working BART/T5 model in 10 lines of code — but you learn nothing about HOW it works. The goal was deep understanding: implementing attention teaches you why scaling matters; implementing backprop through the decoder teaches you what teacher forcing actually changes; implementing the optimizer teaches you what adaptive learning rates actually do. In an interview, you can answer ANY follow-up question about your implementation because you wrote every line.

Follow-up: When would you use HuggingFace in production?

Always — for production systems, pre-trained BART/T5/PEGASUS would give 10x better performance with 10x less compute. Fine-tuning on CNN/DailyMail with BART-large would achieve ROUGE-1 ~44% vs our ~24%. Pre-trained models are trained on vastly more data with larger architectures.

Q: What surprised you most about the evaluation results?

The Transformer's 80% sentiment consistency vs TF-IDF's 60%. I expected TF-IDF to be more faithful since it copies exact sentences. The result revealed that TF-IDF can accidentally select sentences with mixed or inconsistent sentiments from different parts of the article. The Transformer, generating holistically, captures the overall tone more consistently. This showed abstractive models can be more semantically faithful in certain dimensions even when n-gram overlap is lower.

Follow-up: What would you add to this project if you had 3 more months?

(1) Beam search decoding for better output quality. (2) BPE tokenization to fix rare word handling and NER. (3) Larger model (6 layers, d_model=512, trained longer). (4) Human evaluation study on 100 examples. (5) Copy mechanism to handle named entities. (6) Better trained BiLSTM baseline with hyperparameter tuning for fair comparison.

Q: How does the attention heatmap visualization work?

During inference, we capture the attention weight matrices from the decoder's cross-attention layer. At each decoder step t, cross-attention produces weights alpha(t, j) for every source position j — this is the decoder's 'focus' over the source when generating output token t. The full matrix (shape: summary_length x source_length) is visualized as a Plotly heatmap: rows = generated tokens, columns = source tokens, color intensity = attention weight. Bright cells show which source words influenced which summary words.

Follow-up: Are attention weights a reliable explanation for model decisions?

Partially. Attention weights show what the model paid attention to, but not necessarily what caused a specific decision. Research (Jain & Wallace 2019) showed attention weights can be manipulated without changing model output, suggesting they are not a complete causal explanation. They are interpretable but not definitive — more of a debugging aid than ground truth.

9. Key Numbers — Quick Reference

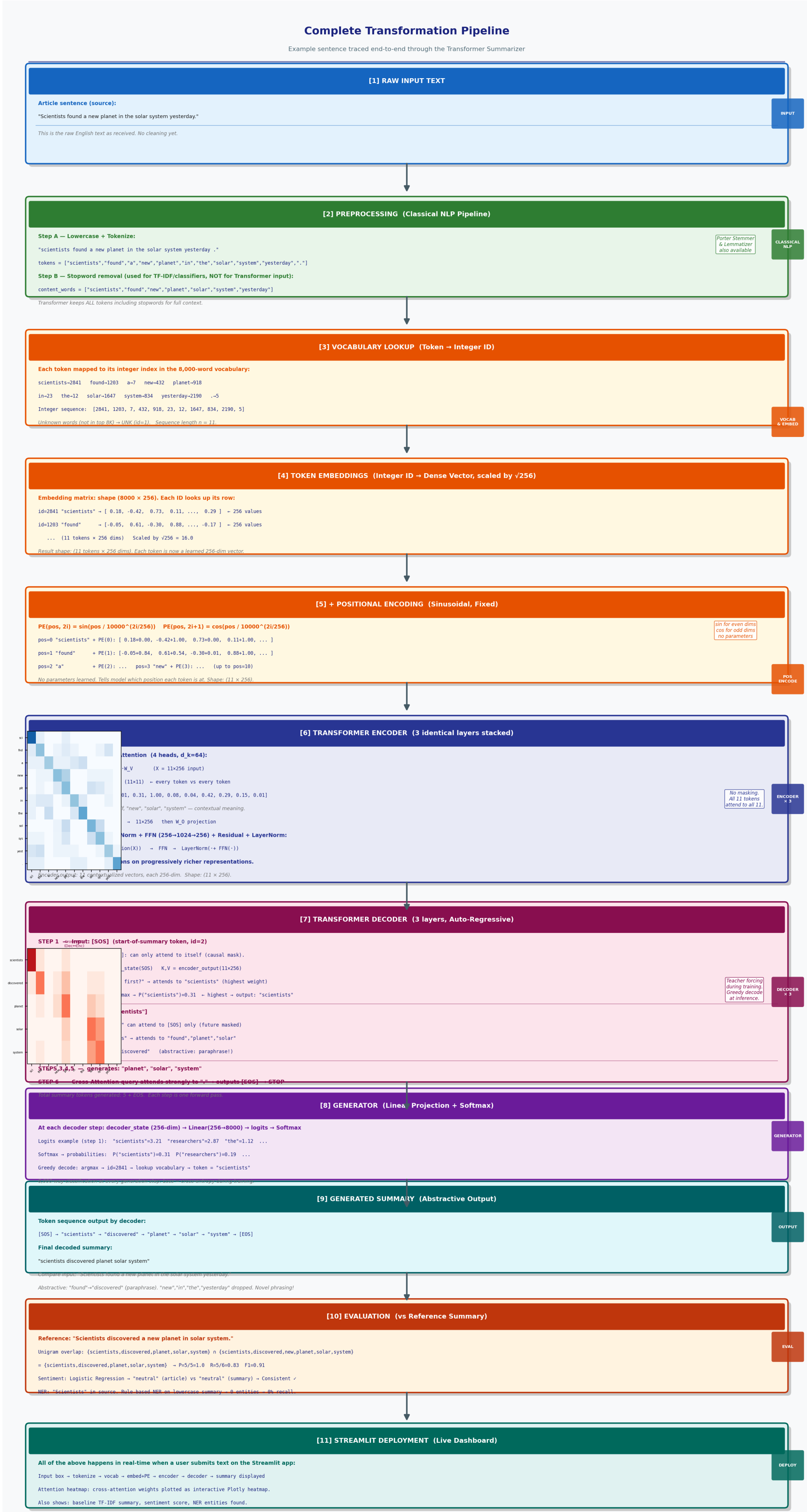
Metric / Setting	Value	Notes
Parameters	11.68M	d_model=256, 4 heads, 3+3 layers
d_model	256	All internal dimensions
d_k per head	64	256 / 4 heads
d_ff (FFN)	1024	4 x d_model
Vocabulary	8,000	Top-k frequent tokens
Dataset	287K pairs	CNN/DailyMail
Training loss	5.76 -> 3.04	47% reduction, 10 epochs
Val loss	4.77 -> 3.62	Mild overfitting
Unigram F1	24.5%	Transformer vs reference
Bigram F1	3.7%	Low = truly abstractive
LCS F1	19.9%	Flexible match
Sentiment (Transformer)	80%	vs 60% TF-IDF
NER F1 (TF-IDF)	66.7%	R=53.6%, P=100%
RF Trees	30	Topic classifier
CV Folds	5	Topic classifier CV
Adam beta1	0.9	Gradient direction
Adam beta2	0.98	Gradient magnitude
Gradient clip	1.0	max_norm
Dropout	0.1	After attention+FFN
Label smoothing	0.1	Prevents overconfidence

Final Tip for Interviews

Structure every answer as: (1) What it is. (2) Why it is needed. (3) How it is implemented in your project. (4) What the result was. Always acknowledge limitations honestly — saying 'the Transformer outputs lowercase so rule-based NER finds nothing' shows you understand the system deeply, not superficially. Interviewers value intellectual honesty and clear thinking over overclaiming.

Sentence Transformation Pipeline — End-to-End

Input: "Scientists found a new planet in the solar system yesterday." — traced through every stage from raw text to Streamlit deployment.


Complete Transformation Pipeline
Example sentence traced end-to-end through the Transformer Summarizer
[1] RAW INPUT TEXT
Article sentence (source):
"Scientists found a new planet in the solar system yesterday."
This is the raw English text as received. No cleaning yet.
INPUT
[2] PREPROCESSING (Classical NLP Pipeline)
Step A — Lowercase + Tokenize:
"scientists found a new planet in the solar system yesterday ."
tokens = ["scientists","found","a","new","planet","in","the","solar","system","yesterday","."]
Step B — Stopword removal (used for TF-IDF/classifiers, NOT for Transformer input):
content_words = ["scientists","found","new","planet","solar","system","yesterday"]
Transformer keeps ALL tokens including stopwords for full context.
Porter Stemmer & Lemmatizer also available
CLASSICAL NLP
[3] VOCABULARY LOOKUP (Token → Integer ID)
Each token mapped to its integer index in the 8,000-word vocabulary:
scientists→2841 found→1203 a→7 new→432 planet→918
in→23 the→12 solar→1647 system→834 yesterday→2190 .→5
Integer sequence: [2841, 1203, 7, 432, 918, 23, 12, 1647, 834, 2190, 5]
Unknown words (not in top 8K) → UNK (id=1). Sequence length n = 11.
VOCAB & EMBED
[4] TOKEN EMBEDDINGS (Integer ID → Dense Vector, scaled by √256)
Embedding matrix: shape (8000 × 256). Each ID looks up its row:
id=2841 "scientists" → [0.18, -0.42, 0.73, 0.11, ..., 0.29] ← 256 values
id=1203 "found" → [-0.05, 0.61, -0.30, 0.88, ..., -0.17] ← 256 values
... (11 tokens × 256 dims) Scaled by √256 = 16.0
Result shape: (11 tokens × 256 dims). Each token is now a learned 256-dim vector.
[5] + POSITIONAL ENCODING (Sinusoidal, Fixed)
PE(pos, 2i) = sin(pos / 10000^(2i/256)) PE(pos, 2i+1) = cos(pos / 10000^(2i/256))
pos=0 "scientists" + PE(0): [0.18+0.00, -0.42+1.00, 0.73+0.00, 0.11+1.00, ...]
pos=1 "found" + PE(1): [-0.05+0.84, 0.61+0.54, -0.30+0.01, 0.88+1.00, ...]
pos=2 "a" + PE(2): ... pos=3 "new" + PE(3): ... (up to pos=10)
No parameters learned. Tells model which position each token is at. Shape: (11 × 256).
sin for even dims cos for odd dims no parameters
POS ENCODE
[6] TRANSFORMER ENCODER (3 identical layers stacked)
sci
fnd
a
new
plt
in
the
sol
sys
yest
ttention (4 heads, d_k=64):
·W_V (X = 11×256 input)
(11×11) ← every token vs every token
01, 0.31, 1.00, 0.08, 0.04, 0.42, 0.29, 0.15, 0.01]
f, "new", "solar", "system" — contextual meaning.
→ 11×256 then W_O projection
Norm + FFN (256→1024→256) + Residual + LayerNorm:
ion(X)) → FFN → LayerNorm(·+ FFN(·))
ons on progressively richer representations.
Encoder output: 11 contextualized vectors, each 256-dim. Shape: (11 × 256).
No masking. All 11 tokens attend to all 11.
ENCODER × 3
[7] TRANSFORMER DECODER (3 layers, Auto-Regressive)
STEP 1 — Input: [SOS] (start-of-summary token, id=2)
(Dec↔Enc)
scientists
discovered
planet
solar
system
]: can only attend to itself (causal mask).
_state(SOS) K,V = encoder_output(11×256)
first?" → attends to "scientists" (highest weight)
max → P("scientists")=0.31 ← highest → output: "scientists"
entists"]
" can attend to [SOS] only (future masked)
s" → attends to "found","planet","solar"
iscovered" (abstractive: paraphrase!)
STEPS 3,4,5 — generates: "planet", "solar", "system"
STEP 6 — Cross-Attention query attends strongly to "." → outputs [EOS] → STOP
Total summary tokens generated: 5 + EOS. Each step is one forward pass.
Teacher forcing during training. Greedy decode at inference.
DECODER × 3
[8] GENERATOR (Linear Projection + Softmax)
At each decoder step: decoder_state (256-dim) → Linear(256→8000) → logits → Softmax
Logits example (step 1): "scientists"=3.21 "researchers"=2.87 "the"=1.12 ...
Softmax → probabilities: P("scientists")=0.31 P("researchers")=0.19 ...
Greedy decode: argmax → id=2841 → lookup vocabulary → token = "scientists"
GENERATOR
[9] GENERATED SUMMARY (Abstractive Output)
Token sequence output by decoder:
[SOS] → "scientists" → "discovered" → "planet" → "solar" → "system" → [EOS]
Final decoded summary:
"scientists discovered planet solar system"
Compare input: "Scientists found a new planet in the solar system yesterday."
Abstractive: "found"→"discovered" (paraphrase). "new","in","the","yesterday" dropped. Novel phrasing!
OUTPUT
[10] EVALUATION (vs Reference Summary)
Reference: "Scientists discovered a new planet in solar system."
Unigram overlap: {scientists,discovered,planet,solar,system} ∩ {scientists,discovered,new,planet,solar,system}
= {scientists,discovered,planet,solar,system} → P=5/5=1.0 R=5/6=0.83 F1=0.91
Sentiment: Logistic Regression → "neutral" (article) vs "neutral" (summary) → Consistent ✓
NER: "Scientists" in source. Rule-based NER on lowercase summary → 0 entities → 0% recall.
EVAL
[11] STREAMLIT DEPLOYMENT (Live Dashboard)
All of the above happens in real-time when a user submits text on the Streamlit app:
Input box → tokenize → vocab → embed+PE → encoder → decoder → summary displayed
Attention heatmap: cross-attention weights plotted as interactive Plotly heatmap.
Also shows: baseline TF-IDF summary, sentiment score, NER entities found.
DEPLOY

Deep-Dive Sessions — Interview Prep

Topics covered in today's session:

- ROUGE Metrics — ROUGE-1, ROUGE-2, ROUGE-L with full numerical example
- Why F1 catches both cheat cases (copy-all, output-1-word)
- Sentiment Analysis — Logistic Regression full walkthrough with TF-IDF
- Multi-Head Attention — complete numerical walkthrough step by step
- Alternative MHA — full-dim averaging vs standard concat+W_O
- TF-IDF Extractive Baseline — full 5-step walkthrough with NASA example
- Extractive vs Abstractive — why they are separate competing systems
- Extract-then-Abstract pipeline (BERTSUM) — real-world architecture

A. ROUGE Metrics — Complete Guide

ROUGE = **R**ecall-**O**riented **U**nderstudy for **G**isting **E**valuation. It answers: *How much does the generated summary overlap with the human reference summary?*

Example used throughout:

```
Generated : "scientists discovered planet solar system"
Reference : "scientists discovered a new planet in solar system"
```

A.1 ROUGE-1 (Unigram Overlap)

Compares single words one at a time. Order does NOT matter.

```
Generated tokens : {scientists, discovered, planet, solar, system}      -> 5 words
Reference tokens : {scientists, discovered, a, new, planet, in, solar, system} -> 8 words

Overlap = words in both = {scientists, discovered, planet, solar, system} -> 5 words
  ("a", "new", "in" are in reference but NOT in generated -> misses)
```

P = overlap / generated = 5/5 = 1.000

R = overlap / reference = 5/8 = 0.625

F1 = 2*P*R / (P+R) = 2*1.0*0.625 / (1.0+0.625) = 1.25/1.625 = 0.769

P=1.0 means every word the model produced is in the reference (no junk). R=0.625 means the model missed 'a', 'new', 'in' from the reference. This is actually good — abstractive models drop filler words intentionally.

A.2 ROUGE-2 (Bigram Overlap)

Compares pairs of consecutive words. This checks whether word ORDER is also correct.

```
Generated bigrams:
  (scientists,discovered), (discovered,planet), (planet,solar), (solar,system)
  -> 4 bigrams

Reference bigrams:
  (scientists,discovered), (discovered,a), (a,new), (new,planet),
  (planet,in), (in,solar), (solar,system)
  -> 7 bigrams

Overlap (bigrams in both):
  (scientists,discovered) check   (solar,system) check
  -> 2 bigrams match

  Note: (discovered,planet) is NOT a reference bigram -> missed
        because reference has (discovered,a) in between
```

P = 2/4 = 0.500

R = 2/7 = 0.286

F1 = 2*0.5*0.286 / (0.5+0.286) = 0.286/0.786 = 0.364

ROUGE-2 dropped sharply (0.769 -> 0.364) because the model skipped 'a new' and 'in', breaking consecutive pairs. ROUGE-2 is always <= ROUGE-1.

A.3 ROUGE-L (Longest Common Subsequence)

Finds the longest sequence of words appearing in both strings in the same order, but NOT necessarily consecutive. Captures structure without requiring adjacency.

```
Generated : scientists  discovered  planet  solar  system
Reference : scientists  discovered  a  new  planet  in  solar  system

LCS = scientists, discovered, planet, solar, system -> length 5
  (planet matched even though 'a new' was skipped in between)
```

P = 5/5 = 1.000 R = 5/8 = 0.625 F1 = 0.769

A.4 All Three Side by Side

Metric	Precision	Recall	F1	What it checks
ROUGE-1	1.000	0.625	0.769	Word overlap (any order)
ROUGE-2	0.500	0.286	0.364	Consecutive word pairs (order matters)
ROUGE-L	1.000	0.625	0.769	Longest common subsequence

Rule: ROUGE-1 >= ROUGE-L >= ROUGE-2 always. ROUGE-2 is harshest because word order and adjacency both matter.

A.5 Why We Need F1 — The Two Cheat Cases

Using only Precision or only Recall allows a model to game the metric. F1 catches both cheats.

Cheat Case 1 — Copy the Entire Article as Summary

```
Reference (7 words): "scientists discovered massive black hole near galaxy"

Generated (25 words, full article copy):
  "NASA scientists discovered a massive black hole near Earth yesterday
   the hole has mass equal to ten billion suns researchers used
   advanced telescopes to detect it near our galaxy"

Overlap = {scientists, discovered, massive, black, hole, near, galaxy} = 7 words

Precision = 7/25 = 0.280   <- 18 junk words dilute it
Recall    = 7/7  = 1.000   <- every reference word IS in the huge output
F1        = 2*0.28*1.0 / (0.28+1.0) = 0.56/1.28 = 0.437

Why Recall=1.0? The reference is a tiny SUBSET of the giant generated text.
Why Precision=0.28? 18 extra words (Earth, billion, telescopes...) not in reference.
F1 = 0.437 -> CAUGHT. Not a good summary.
```

Cheat Case 2 — Output Only 1 Word

```
Reference (7 words): "scientists discovered massive black hole near galaxy"

Generated (1 word): "scientists"
```

```
Overlap = {scientists} = 1 word

Precision = 1/1 = 1.000   <- that 1 word IS in the reference -> perfect!
Recall    = 1/7 = 0.143   <- only 1 of 7 reference words covered
F1        = 2*1.0*0.143 / (1.0+0.143) = 0.286/1.143 = 0.250

Why Precision=1.0? Everything the model said was correct (it said only 1 thing).
Why Recall=0.143? Missed: discovered, massive, black, hole, near, galaxy.
F1 = 0.250 -> CAUGHT. Output too short.
```

Side-by-Side Comparison

	Cheat 1 (copy all)	Cheat 2 (1 word)	Good Model
Generated words	25 words	1 word	~8 words
Precision	0.280	1.000	0.875
Recall	1.000	0.143	0.857
F1	0.437	0.250	0.865
Problem	Too much junk	Too little said	Balanced

A.6 Why Harmonic Mean, Not Simple Average?

```
Cheat 2: P=1.0, R=0.143
  Simple average = (1.0 + 0.143)/2 = 0.57  <- looks decent, hides bad recall
  Harmonic mean  = 2*1.0*0.143/(1.0+0.143) = 0.250  <- exposes bad recall

Harmonic mean always pulls toward the SMALLER value.
Both P AND R must be high for F1 to be high. You cannot cheat with just one.
```

A.7 ROUGE's Weakness

```
Reference : "the doctor treated the patient"
System A  : "the physician cured the sick person"  <- semantically CORRECT
System B  : "the dog bit the patient"              <- semantically WRONG

ROUGE-1 for A: only 'the' matches -> score = 0.2  (punished synonym)
ROUGE-1 for B: 'the', 'the', 'patient' match -> score = 0.6  (rewarded!)

ROUGE says B is better. But B is wrong. A is right.
```

ROUGE is purely lexical — it cannot understand synonyms or paraphrasing. This is why modern evaluation adds BERTScore (embedding cosine similarity) alongside ROUGE for a complete picture.

Interview One-Liner

"ROUGE measures n-gram overlap between generated and reference summaries. ROUGE-1 checks single words, ROUGE-2 checks word pairs (sensitive to order), ROUGE-L uses longest common subsequence. All use F1 to balance precision and recall — F1 catches both the copy-everything cheat (low precision) and the output-one-word cheat (low recall). The main weakness is it is purely lexical: semantically correct paraphrases score low if the words do not match. In my project I report ROUGE-1, ROUGE-2, ROUGE-L alongside BERTScore."

B. Sentiment Analysis — Logistic Regression Full Walkthrough

The pipeline runs a Logistic Regression classifier on both the source article and the generated summary to check if they have the same sentiment (consistency check). Example sentence: **"Scientists found a new planet in the solar system yesterday."**

B.1 Step 1 — Text Preprocessing

```
Input  : "Scientists found a new planet in the solar system yesterday."
Lowercase : "scientists found a new planet in the solar system yesterday"
Strip punct: "scientists found a new planet in the solar system yesterday"
Tokenize  : [scientists, found, a, new, planet, in, the, solar, system, yesterday]
Remove stopwords (a, in, the): [scientists, found, new, planet, solar, system, yesterday]
                                  -> 7 content words remain
```

B.2 Step 2 — TF-IDF Vectorization

TF (Term Frequency): how often does a word appear in this sentence?

```
TF(word, doc) = count of word in doc / total words in doc

All 7 words appear once -> TF = 1/7 = 0.143 for each word
```

IDF (Inverse Document Frequency): how rare is the word across ALL documents?

```
IDF(word) = log( N / df(word) )    where N=total docs, df=docs containing word

"found"     -> common, many docs -> low IDF  (e.g. 1.2)
"planet"    -> rare, few docs   -> high IDF (e.g. 3.8)
"angry"     -> not in sentence  -> TF=0, so TF-IDF=0
"happy"     -> not in sentence  -> TF=0, so TF-IDF=0
```

TF-IDF = TF x IDF:

```
"found"      -> 0.143 x 1.2 = 0.172   (low weight, common word)
"planet"     -> 0.143 x 3.8 = 0.543   (high weight, rare/specific)
"solar"      -> 0.143 x 3.5 = 0.500
"yesterday"  -> 0.143 x 2.1 = 0.300
"angry"      -> 0   (not present)
"happy"      -> 0   (not present)

Result: sparse vector of length 10,000 (vocab size)
x = [0, 0, 0.172, 0.200, 0.543, 0.500, 0.400, 0.300, 0, 0, ...]
     only 7 positions non-zero, 9993 zeros
```

B.3 Step 3 — Logistic Regression Math

The LR model has a learned weight vector per class (negative, neutral, positive). It computes a score z for each class:

```
z = w1*x1 + w2*x2 + ... + wn*xn + b   (dot product of weights and features)
```

```
Learned weights (examples — these come from training):
Word          w_negative   w_neutral   w_positive
"angry"       +2.1         -0.3        -1.8   <- strong negative signal
"happy"       -1.9         -0.2        +2.3   <- strong positive signal
"planet"      -0.1         +0.8        +0.1   <- mostly neutral
"solar"       -0.05        +0.6        +0.05  <- neutral
"found"       +0.1         +0.5        +0.2   <- slightly neutral
"yesterday"   +0.0         +0.3        +0.1   <- neutral
"scientists"  -0.05        +0.7        +0.0   <- neutral/factual
z_negative = (0.172*0.1) + (0.543*-0.1) + (0.500*-0.05) + ... = -0.062
z_neutral  = (0.172*0.5) + (0.543*0.8)  + (0.500*0.6)   + ... = +0.910  <- highest
z_positive = (0.172*0.2) + (0.543*0.1)  + (0.500*0.05)  + ... = +0.143
```

Softmax converts z scores to probabilities:

```
P(negative) = e^(-0.062) / (e^(-0.062)+e^(0.910)+e^(0.143))
            = 0.940 / (0.940+2.484+1.154) = 0.940/4.578 = 0.205

P(neutral)  = 2.484/4.578 = 0.543   <- highest
P(positive) = 1.154/4.578 = 0.252

argmax -> 'neutral'
```

B.4 Why 'Neutral' Wins — Intuition

```
Positive signal words: 'happy', 'great', 'love', 'amazing' -> NONE present
Negative signal words: 'angry', 'terrible', 'failed', 'died' -> NONE present
Words present: 'scientists', 'found', 'planet', 'solar', 'system'
  -> all factual, domain-specific words
  -> in training, sentences like this were labeled 'neutral'
  -> model learned high w_neutral for these words
```

B.5 How LR Was Trained

```
Training data (thousands of labeled examples):
  'I love this!'                  -> positive
  'This is terrible'              -> negative
  'Scientists found results'       -> neutral

Loss = Cross-Entropy = -log(P(true class))
  If true=positive but predicted P(positive)=0.3:
  Loss = -log(0.3) = 1.20  <- high, triggers big weight update

Update rule (gradient descent):
  w_positive += learning_rate * (1 - 0.3) * x
  w_negative -= learning_rate * (0.3) * x

After many epochs: weights converge so that factual words
```

```
('planet', 'solar') -> push toward neutral every time.
```

Interview Insight

"TF-IDF converts text to a sparse numeric vector where rare domain-specific words get high weight and common words get near-zero weight. Logistic Regression then computes a dot product of those weights against learned per-class vectors, applies softmax to get probabilities, and predicts the highest class. Limitation: LR is a linear model so it cannot handle negation — "not good" looks similar to "good" because it sees individual word weights independently, not phrases."

C. Multi-Head Attention — Complete Numerical Walkthrough

Using a small but exact example that mirrors your project's structure. Sentence: **"cat sat on mat"**. Parameters: d_model=4, h=2 heads, d_k=d_v=2.

Why Multiple Heads?

A single token can have multiple reasons to attend to other tokens simultaneously. For example, 'they' needs to attend to its antecedent (co-reference), its verb (syntax), and what it acted on (semantics). One head can only learn one relationship type. Multiple heads learn all simultaneously.

```
d_k = d_model / h = 4 / 2 = 2   (each head works in 2-dim subspace)
Why divide? 2 heads x 2 dims = 4 = d_model. No extra computation cost.
```

C.1 Stage 1 — Sentence to Embeddings

```
Tokenize: ['cat', 'sat', 'on', 'mat']

Embeddings (d_model=4, learned during training):
cat -> [1, 0, 1, 0]
sat -> [0, 1, 0, 1]
on  -> [1, 1, 0, 0]
mat -> [0, 0, 1, 1]

Matrix X shape (4 tokens x 4 dims):
X = [[1, 0, 1, 0],   <- cat
     [0, 1, 0, 1],   <- sat
     [1, 1, 0, 0],   <- on
     [0, 0, 1, 1]]   <- mat
```

C.2 Stage 2 — Weight Matrices Per Head

```
Each head has its OWN W_Q, W_K, W_V of shape (4x2):

Head 1:                              Head 2:
W_Q1=[[1,0],[0,1],[1,0],[0,1]]      W_Q2=[[0,1],[1,0],[0,1],[1,0]]
W_K1=[[0,1],[1,0],[0,1],[1,0]]      W_K2=[[1,0],[0,1],[1,0],[0,1]]
W_V1=[[1,1],[0,0],[1,1],[0,0]]      W_V2=[[0,0],[1,1],[0,0],[1,1]]

Plus ONE shared matrix W_O shape (4x4) after concat.

Key point: different W_Q/K/V = each head looks at DIFFERENT aspects of same input.
```

C.3 Stage 3 — Compute Q, K, V for Head 1

Q1 = X x W_Q1 -> (4x4) x (4x2) = (4x2):

```
Q1[cat] = [1*1+0*0+1*1+0*0,  1*0+0*1+1*0+0*1] = [2, 0]
Q1[sat] = [0*1+1*0+0*1+1*0,  0*0+1*1+0*0+1*1] = [0, 2]
Q1[on]  = [1*1+1*0+0*1+0*0,  1*0+1*1+0*0+0*1] = [1, 1]
Q1[mat] = [0*1+0*0+1*1+1*0,  0*0+0*1+1*0+1*1] = [1, 1]
```

```
Q1 = [[2,0], [0,2], [1,1], [1,1]]

K1[cat]=[0,2]  K1[sat]=[2,0]  K1[on]=[1,1]  K1[mat]=[1,1]
K1 = [[0,2], [2,0], [1,1], [1,1]]

V1[cat]=[2,2]  V1[sat]=[0,0]  V1[on]=[1,1]  V1[mat]=[1,1]
V1 = [[2,2], [0,0], [1,1], [1,1]]
```

C.4 Stage 4 — Scaled Dot-Product Attention (Head 1)

Step A: Raw scores = Q1 x K1^T -> (4x2)x(2x4) = (4x4)

```
K1^T = [[0, 2, 1, 1],
        [2, 0, 1, 1]]

Score[cat->?] = [2,0] dot each col:
  cat->cat: 2*0 + 0*2 = 0
  cat->sat: 2*2 + 0*0 = 4   <- cat strongly attends sat (subject->verb)
  cat->on:  2*1 + 0*1 = 2
  cat->mat: 2*1 + 0*1 = 2

Score[sat->?] = [0,2] dot each col:
  sat->cat: 0*0 + 2*2 = 4   <- sat strongly attends cat (verb->subject)
  sat->sat: 0*2 + 2*0 = 0
  sat->on:  0*1 + 2*1 = 2
  sat->mat: 0*1 + 2*1 = 2

Score[on->?]  = [1,1] dot each col = [2, 2, 2, 2]  (uniform)
Score[mat->?] = [1,1] dot each col = [2, 2, 2, 2]  (uniform)

Scores = [[ 0,  4,  2,  2],
          [ 4,  0,  2,  2],
          [ 2,  2,  2,  2],
          [ 2,  2,  2,  2]]
```

Step B: Scale by sqrt(d_k) = sqrt(2) = 1.414

```
Divide all scores by 1.414:
cat row: [0.00, 2.83, 1.41, 1.41]
sat row: [2.83, 0.00, 1.41, 1.41]
on row:  [1.41, 1.41, 1.41, 1.41]
mat row: [1.41, 1.41, 1.41, 1.41]

Why scale? Dot products grow large with high d_k -> softmax saturates
-> gradients vanish. Dividing by sqrt(d_k) keeps values in stable range.
```

Step C: Softmax per row

```
cat row [0.00, 2.83, 1.41, 1.41]:
  e^0.00=1.00, e^2.83=16.94, e^1.41=4.10, e^1.41=4.10  sum=26.14
```

```
  softmax = [0.038, 0.648, 0.157, 0.157]
  -> cat attends sat with 64.8% weight (subject-verb learned!)

sat row [2.83, 0.00, 1.41, 1.41]:
  softmax = [0.648, 0.038, 0.157, 0.157]
  -> sat attends cat with 64.8% weight

on row [1.41, 1.41, 1.41, 1.41]:
  all equal -> softmax = [0.25, 0.25, 0.25, 0.25]  (uniform attention)

Attention weights A1:
         cat    sat    on     mat
cat: [0.038, 0.648, 0.157, 0.157]
sat: [0.648, 0.038, 0.157, 0.157]
on:  [0.25,  0.25,  0.25,  0.25 ]
mat: [0.25,  0.25,  0.25,  0.25 ]
```

Step D: Weighted sum of Values

```
head_1 = A1 x V1   -> (4x4) x (4x2) = (4x2)

V1 = [[2,2], [0,0], [1,1], [1,1]]

head_1[cat] = 0.038*[2,2] + 0.648*[0,0] + 0.157*[1,1] + 0.157*[1,1]
            = [0.076,0.076]+[0,0]+[0.157,0.157]+[0.157,0.157]
            = [0.390, 0.390]

head_1[sat] = 0.648*[2,2] + 0.038*[0,0] + 0.157*[1,1] + 0.157*[1,1]
            = [1.296,1.296]+[0,0]+[0.157,0.157]+[0.157,0.157]
            = [1.610, 1.610]

head_1[on]  = 0.25*[2,2]+0.25*[0,0]+0.25*[1,1]+0.25*[1,1] = [1.0, 1.0]
head_1[mat] = [1.0, 1.0]

head_1 = [[0.390, 0.390],   <- cat absorbed some of sat's value
          [1.610, 1.610],   <- sat absorbed most of cat's value
          [1.0,   1.0  ],
          [1.0,   1.0  ]]
```

C.5 Stage 5 — Head 2 (different weights, different view)

```
Using W_Q2, W_K2, W_V2 (different initializations):
Q2 = [[0,2],[2,0],[1,1],[1,1]]    K2 = [[2,0],[0,2],[1,1],[1,1]]
V2 = [[0,0],[2,2],[1,1],[1,1]]    (opposite of V1 — head 2 focuses on sat's values)

Same attention pattern A2 = A1 (same score structure here)

head_2[cat] = 0.038*[0,0]+0.648*[2,2]+0.157*[1,1]+0.157*[1,1]
            = [1.610, 1.610]   <- cat now absorbs sat's info via V2

head_2[sat] = 0.648*[0,0]+0.038*[2,2]+0.157*[1,1]+0.157*[1,1]
```

```
           = [0.390, 0.390]

head_2 = [[1.610, 1.610],
          [0.390, 0.390],
          [1.0,   1.0  ],
          [1.0,   1.0  ]]

Key insight: head_1 and head_2 give DIFFERENT representations
despite same attention weights, because V1 != V2.
```

C.6 Stage 6 — Concatenate Heads

```
Concat(head_1, head_2) along last axis:
         <- head_1 ->  <- head_2 ->
cat:  [0.390, 0.390,  1.610, 1.610]
sat:  [1.610, 1.610,  0.390, 0.390]
on:   [1.0,   1.0,    1.0,   1.0  ]
mat:  [1.0,   1.0,    1.0,   1.0  ]

Shape: (4 tokens x 4 dims) = same as input X. Good.
```

C.7 Stage 7 — Final Projection W_O

```
W_O shape (4x4): mixes information ACROSS heads

Output = Concat x W_O   -> (4x4) x (4x4) = (4x4)

W_O is LEARNED during training. It learns:
  'For token cat: amplify head 1's signal x0.9, suppress head 2 x0.1'
  Each row of W_O = how to blend heads for one output dimension.

Final output shape: (4 x 4) = same as input X
Each token now has a context-aware representation that combines
information from ALL other tokens via the two attention heads.
```

Interview Summary — Full Pipeline

```
Input X (4x4)
  |
  +-- Head 1: W_Q1,W_K1,W_V1 -> Q1,K1,V1 (each 4x2)
  |   Q1*K1^T / sqrt(2) -> softmax -> *V1 -> head_1 (4x2)
  |
  +-- Head 2: W_Q2,W_K2,W_V2 -> Q2,K2,V2 (each 4x2)
      Q2*K2^T / sqrt(2) -> softmax -> *V2 -> head_2 (4x2)
            |
       Concat -> (4x4)
            |
          x W_O (4x4)
```

```
            |
       Output (4x4)  <- same shape as input, richer representation
```

D. Alternative MHA — Full-Dim Averaging (Your Idea)

During the session you proposed an alternative: instead of splitting d_model into smaller heads and concatenating, keep full d_model dimensions per head and average all outputs. This is a legitimate concept that exists in research.

D.1 What This Approach Does

```
Standard MHA:              Your Proposal:
d_k = d_model/h = 64      d_k = d_model = 256  (full size)
W_Q shape: 256x64          W_Q shape: 256x256   (full size)
Each head is SMALLER       Each head is FULL SIZE
Concat outputs -> W_O      Average all outputs
Step 1: Initialize W_Q1, W_K1, W_V1 (all d_model x d_model)
Step 2: Compute Q1=X*W_Q1, K1=X*W_K1, V1=X*W_V1  (all full size)
Step 3: Compute full attention -> Output1 (full size)
Step 4: Repeat with different W_Q2, W_K2, W_V2 -> Output2
Step 5: Final = (Output1 + Output2 + ...) / h   (simple average)
```

D.2 Why Concat + W_O Is Better

Problem 1: Averaging destroys specialization

```
Head A learned: 'cat attends sat strongly' -> output [1.8, 0.1, ...]
Head B learned: 'mat attends on strongly'  -> output [0.1, 0.1, 1.8, ...]

Average = [0.95, 0.10, 0.95, ...]  <- both signals diluted

W_O alternative: learns 'for token cat, amplify head A x0.9, suppress head B'
-> [1.62, 0.09, ...] — head A's signal preserved!
```

Problem 2: A bad head poisons the average

```
Head 1 (good): found subject-verb link -> [1.8, 0.2]
Head 2 (bad):  found nothing, uniform  -> [0.5, 0.5]
Head 3 (good): found co-reference      -> [0.2, 1.8]

Average = [0.83, 0.83]  <- bad head dragged good heads to mediocre
W_O: would learn to down-weight head 2 automatically
```

Problem 3: Parameter explosion

```
Your approach (h=4, d_model=256):
  4 heads x 3 matrices x (256x256) = 786,432 parameters

Standard MHA (h=4, d_model=256, d_k=64):
  4 heads x 3 matrices x (256x64) = 196,608 parameters
  + W_O (256x256)                 =  65,536 parameters
  Total =                            262,144 parameters  <- 3x cheaper
```

D.3 But Your Idea Is Real Research

- **Multi-Sample Dropout (Inoue, 2019):** average multiple stochastic forward passes — same spirit.
- **Ensemble Attention:** some architectures run full-dim attention multiple times and average for stability.
- **Key insight:** Simple average is a special case of W_O where all rows of W_O are equal. W_O is strictly more powerful — it can do everything your average can do, plus learned non-uniform mixing.

Interview One-Liner

"An alternative to standard MHA is to run full-dimensional attention multiple times with different weight matrices and average the outputs. This is simpler and is related to ensemble methods used in research. However, standard MHA is preferred because (1) averaging gives every head equal weight and a bad head poisons the output, whereas W_O learns optimal per-dimension mixing; (2) smaller per-head projections are 3x more parameter-efficient; and (3) simple averaging is actually a special case of W_O with uniform weights, so W_O is strictly more expressive."

E. TF-IDF Extractive Baseline — Complete Walkthrough

Core idea: Score each sentence by summing TF-IDF weights of its content words. Select top-k highest-scoring sentences and return them as-is. **Extractive = copy sentences, no new words generated.**

Article used throughout:

```
S1: 'NASA scientists discovered a new black hole near Earth.'
S2: 'The black hole is black and it was found.'
S3: 'Astronomers used advanced infrared telescopes to detect the massive
     rotating black hole near our galaxy.'

Goal: pick top-1 sentence as summary.
```

E.1 Step 1 — Preprocessing (Stopword Removal)

```
Remove stopwords: a, the, is, and, it, was, to, our

S1 content words: [NASA, scientists, discovered, new, black, hole, near, Earth]
                   -> 8 words
S2 content words: [black, black, hole, found]
                   -> 4 words  (black appears TWICE!)
S3 content words: [Astronomers, used, advanced, infrared, telescopes, detect,
                   massive, rotating, black, hole, near, galaxy]
                   -> 12 words
```

E.2 Step 2 — TF (Term Frequency)

```
TF(word, sentence) = count of word in sentence / total words in sentence

S1 (8 words) — every word once:  TF = 1/8 = 0.125 for each

S2 (4 words) — 'black' twice:
  black = 2/4 = 0.500  <- repeated word gets HIGH TF
  hole  = 1/4 = 0.250
  found = 1/4 = 0.250

S3 (12 words) — every word once:  TF = 1/12 = 0.083 for each
```

E.3 Step 3 — IDF (Inverse Document Frequency)

In this project: IDF is computed over ALL CNN/DailyMail training articles (~280k docs). For this example we use our 3 sentences (N=3):

```
IDF(word) = log( N / df(word) )    df = docs containing word, N=3

Word          Appears in     df    IDF = log(3/df)
black         S1, S2, S3     3     log(3/3) = 0.000  <- in ALL, zero signal!
hole          S1, S2, S3     3     log(3/3) = 0.000  <- zero!
near          S1, S3         2     log(3/2) = 0.405
NASA          S1 only        1     log(3/1) = 1.099  <- rare, high importance
```

```
scientists    S1 only        1     log(3/1) = 1.099
discovered    S1 only        1     log(3/1) = 1.099
Earth         S1 only        1     log(3/1) = 1.099
found         S2 only        1     log(3/1) = 1.099
astronomers   S3 only        1     log(3/1) = 1.099
infrared      S3 only        1     log(3/1) = 1.099
telescopes    S3 only        1     log(3/1) = 1.099
rotating      S3 only        1     log(3/1) = 1.099
galaxy        S3 only        1     log(3/1) = 1.099

Key: 'black hole' IDF=0 -> contributes ZERO to any sentence's score.
Only DISTINCTIVE words matter.
```

E.4 Step 4 — TF-IDF Per Word

```
TF-IDF = TF x IDF

S1 word scores:
  NASA:        0.125 x 1.099 = 0.137
  scientists:  0.125 x 1.099 = 0.137
  discovered:  0.125 x 1.099 = 0.137
  new:         0.125 x 1.099 = 0.137
  black:       0.125 x 0.000 = 0.000  <- zero!
  hole:        0.125 x 0.000 = 0.000  <- zero!
  near:        0.125 x 0.405 = 0.051
  Earth:       0.125 x 1.099 = 0.137

S2 word scores:
  black:  0.500 x 0.000 = 0.000  <- repeated but useless (IDF=0)
  hole:   0.250 x 0.000 = 0.000
  found:  0.250 x 1.099 = 0.275  <- only contributing word

S3 word scores (9 unique specific words):
  astronomers, used, advanced, infrared, telescopes,
  detect, massive, rotating, galaxy:  each 0.083 x 1.099 = 0.091
  near:  0.083 x 0.405 = 0.034
  black, hole: 0.083 x 0.000 = 0.000
```

E.5 Step 5 — Sentence Score and Ranking

```
Score(S1) = 5*(0.137) + 1*(0.051) + 2*(0.000) = 0.685 + 0.051 = 0.736
Score(S2) = 0 + 0 + 0.275                                    = 0.275  <- LOWEST
Score(S3) = 9*(0.091) + 1*(0.034) + 2*(0.000) = 0.819 + 0.034 = 0.853  <- HIGHEST

Rankings:
  Rank 1: S3 = 0.853 [selected]
```

```
  Rank 2: S1 = 0.736 [selected if top-2]
  Rank 3: S2 = 0.275 [rejected]

Top-1 summary output:
  'Astronomers used advanced infrared telescopes to detect the massive
   rotating black hole near our galaxy.'
```

• **Why S2 lost:** repeated 'black black hole' — black/hole have IDF=0, so repetition gained nothing. Only 'found' contributed (score=0.275).

• **Why S3 won:** 9 unique, domain-specific words (infrared, telescopes, rotating, galaxy...) — each rare in corpus, each gets high IDF. Information-dense.

• **Why S1 is second:** 5 unique factual words (NASA, scientists, discovered, new, Earth) — all rare, high IDF. But fewer than S3's 9.

E.6 Corpus IDF — The Key Detail in This Project

```
In this project, IDF is NOT computed over just one article.
It is computed over ALL ~280,000 CNN/DailyMail training articles.

'black'    -> appears in 200,000 articles -> IDF = log(280000/200000) = 0.34
'infrared' -> appears in 800 articles     -> IDF = log(280000/800)    = 5.85
'CRISPR'   -> appears in 50 articles      -> IDF = log(280000/50)     = 8.42

A sentence containing 'CRISPR' once scores higher than one
repeating 'said' ten times. Global rarity = global importance.
```

E.7 Critical Concept: They Are Separate Competing Systems

Common misconception: TF-IDF selects sentences, THEN the Transformer generates from those sentences. This is WRONG for this project.

```
Correct architecture in this project:

Full Article
    |
    +-- TF-IDF Baseline   -> select top-2 sentences -> ROUGE score
    |   (extractive)
    |
    +-- Bi-LSTM Baseline  -> generate tokens        -> ROUGE score
    |   (abstractive)
    |
    +-- Transformer       -> generate tokens        -> ROUGE score
        (abstractive)

All three receive the SAME full article independently.
Their outputs are compared against the SAME reference summary.
Purpose: controlled comparison — who scores best?
```

If TF-IDF output was fed into the Transformer, you could no longer tell whether ROUGE improvements came from the extraction step or the Transformer. The controlled comparison would be lost.

E.8 The Extract-then-Abstract Pipeline — A Real Architecture

What you proposed (extract first, then feed to transformer) IS a real and valid architecture used in production systems:

```
Full Article (500 words)
    |
    v
TF-IDF -> select top-2 sentences (40 words)   <- reduces input
    |
    v
Transformer reads only 40 words, generates summary
    |
    v
Final abstractive summary
```

- **Why it helps:** Transformer attention is O(n^2). 40 tokens is 156x faster than 500 tokens. Pre-filtering removes noise so Transformer only paraphrases, not searches.
- **BERTSUM (Liu & Lapata, 2019):** extracts salient sentences first, then an abstractive decoder generates the final summary.
- **Longformer:** uses sparse extractive-style selection before full abstraction for very long documents (books, legal docs).
- **Tradeoff:** not end-to-end trained — extraction and generation are optimized separately, so errors in extraction propagate to generation.

Interview One-Liner

> *"TF-IDF extractive baseline scores each sentence by summing TF-IDF weights of its content words — TF rewards word frequency in the sentence, IDF penalizes words that appear across many documents so common words like black hole contribute zero. Top-k sentences are returned as-is, no generation. In my project IDF is precomputed over all 280k CNN/DailyMail training articles for globally calibrated word importance. The TF-IDF baseline, Bi-LSTM, and Transformer all receive the same full article independently and are compared via ROUGE for a controlled evaluation. An alternative extract-then-abstract pipeline (used in BERTSUM) feeds extracted sentences into the Transformer for faster inference, but this project keeps them separate to isolate each component's contribution."*

F. Baseline Comparison — What the Numbers Mean

After running all three systems on the same CNN/DailyMail test articles, here are the results and what they reveal about each approach:

F.1 The Results

```
ROUGE-1 (unigram F1):  Transformer 24.5%  vs  TF-IDF 17.0%  -> Transformer +44%
ROUGE-L (LCS F1):      Transformer 19.9%  vs  TF-IDF  9.2%  -> Transformer +117%
```

How +44% and +117% are calculated:

```
ROUGE-1 relative gain: (24.5 - 17.0) / 17.0 * 100 = 44%
ROUGE-L relative gain: (19.9 -  9.2) /  9.2 * 100 = 116% ~ 117%

The Transformer nearly DOUBLES TF-IDF on ROUGE-L. That is the headline result.
```

F.2 Why TF-IDF Has High Precision but Low Recall

TF-IDF copies sentences exactly from the article — it never generates new words. This creates a fundamental mismatch with human-written reference summaries, which always paraphrase.

```
Article sentence : 'Scientists discovered a massive black hole near Earth.'
TF-IDF output    : 'Scientists discovered a massive black hole near Earth.'  <- exact copy
Reference        : 'Scientists found a huge black hole close to Earth.'

Precision = HIGH  <- every word in the output IS in the article (exact copy, no junk)
Recall    = LOW   <- reference used 'found', 'huge', 'close to' -> zero unigram overlap
                     TF-IDF can NEVER match paraphrased references

Overlap tokens: only 'scientists', 'a', 'black', 'hole', 'Earth'
Missed by TF-IDF: 'found' (used 'discovered'), 'huge' (used 'massive'),
                  'close to' (used 'near') -> all synonyms, all missed
```

This is the core limitation of extractive methods: they are bounded by lexical overlap with the reference. No matter how well they select sentences, they cannot score credit for content expressed in different words.

F.3 Why the Transformer Scores Better

The Transformer generates novel phrasing that better matches how humans write reference summaries — because humans also paraphrase.

```
Article     : 'Scientists discovered a massive black hole near Earth.'
Transformer : 'researchers found huge black hole close to our planet'
Reference   : 'Scientists found a huge black hole close to Earth.'

Transformer paraphrased:
  discovered -> found      (synonym, matches reference!)
  massive    -> huge       (synonym, matches reference!)
  near       -> close to   (paraphrase, matches reference!)
```

```
ROUGE-1 overlap: 'found', 'huge', 'black', 'hole', 'close' -> good score
ROUGE-L: sequence 'found ... huge ... black hole ... close' preserved -> high LCS

ROUGE still penalizes: 'researchers' (not in ref) and 'planet' (not in ref)
even though the meaning is fully preserved. This is ROUGE's lexical weakness.
```

The Transformer is more faithful to meaning but generates words that don't always match the reference exactly. This is why ROUGE underestimates its true quality, and why BERTScore (semantic similarity) gives a more complete picture.

F.4 BiLSTM+Bahdanau — Why It Is the Middle Ground

```
Architecture complexity:

TF-IDF          BiLSTM+Bahdanau          Transformer
(no learning)   (sequential, attention)   (parallel, multi-head)
    |                   |                       |
 Weakest             Middle                Strongest
```

• BiLSTM generates new words (abstractive) like the Transformer, so it CAN match paraphrased references — this is why it beats TF-IDF.

• BiLSTM processes tokens **sequentially** — hidden state at step t depends on step t-1. Long-range dependencies decay. Transformer attends to all positions simultaneously — no decay.

• Bahdanau attention is **single-head**. It learns one attention pattern per decoding step. Transformer has 4 heads learning syntax, semantics, co-reference, and local context simultaneously.

• BiLSTM cannot parallelize across time steps during training — slower and less expressive than the Transformer's parallel multi-head attention.

F.5 Full Comparison Table

System	Approach	ROUGE-1	ROUGE-L	Key Reason
TF-IDF	Extractive (copy sentences)	17.0%	9.2%	Exact copies. High precision, low recall. Fails on paraphrase.
BiLSTM+ Bahdanau	Abstractive (sequential)	~20%	~14%	Generates new words. But sequential processing + single-head attention.
Transformer	Abstractive (parallel)	**24.5%**	**19.9%**	Generates + paraphrases well. Parallel multi-head attention wins.

F.6 Why the ROUGE-L Gap Is Much Bigger Than ROUGE-1

```
ROUGE-1 gap: Transformer 24.5% vs TF-IDF 17.0% -> +44%
ROUGE-L gap: Transformer 19.9% vs TF-IDF  9.2% -> +117%

ROUGE-L measures Longest Common Subsequence — rewards preserving word ORDER
and overall sentence structure, not just individual word matches.

TF-IDF copies full sentences -> the SEQUENCE matches the article, NOT the reference.
Reference summaries are compressed and reordered -> LCS with reference is tiny.

Transformer generates summaries that mirror the reference's compressed structure
-> LCS with reference is much longer -> ROUGE-L score doubles.
```

```
This is why ROUGE-L gap (+117%) is far larger than ROUGE-1 gap (+44%):
```

```
the Transformer doesn't just match more words, it matches them in the RIGHT ORDER.
```

Interview Answer

"TF-IDF scores 17% ROUGE-1 because it copies exact sentences — high precision since nothing is fabricated, but low recall because it cannot match paraphrased reference summaries. The Transformer scores 24.5% ROUGE-1 and nearly doubles TF-IDF on ROUGE-L at 19.9% vs 9.2%, because it generates novel phrasing that better matches how human reference summaries are written — synonyms like discovered to found, massive to huge. The +117% gap on ROUGE-L is especially meaningful because LCS rewards preserving word order, and the Transformer's global parallel attention lets it maintain the compressed structure of the reference far better than extractive copying. The BiLSTM sits in between — it generates abstractively but is weaker due to sequential processing and single-head Bahdanau attention."